MACHINE LEARNING

ASSIGNMENT – 7

1- D
2- A
3- B
4- A
5- C
6- A
7- B
8- B

9. Suppose we have a dataset which have two classes A and B. The percentage of class A is 40% and percentage of class B is 60%. Calculate the Gini index and entropy of the dataset.

Given that
percentage of class A is 40% and
the percentage of class B is 60%,
we can calculate the probability of each class as follows:

Probability of class A: 0.4
Probability of class B: 0.6
Now we can use these probabilities to calculate the Gini index and entropy.

Gini index:
The Gini index measures the impurity of the dataset.
It is calculated as 1 - (sum of squares of probabilities of each class).
Gini index = 1 - (0.4^2 + 0.6^2) = 0.48

Entropy:
Entropy is another measure of impurity. It is calculated as the negative sum of the probabilities of each class multiplied by their log base 2. In this case, the entropy is:

Entropy = - (0.4 * log2(0.4) + 0.6 * log2(0.6)) = 0.971

Therefore, the Gini index of the dataset is 0.48 and the entropy is 0.971.

10. What are the advantages of Random Forests over Decision Tree?

Random forests have several advantages over decision trees:

Reduction in overfitting: Decision trees are prone to overfitting the training data, meaning they can capture noise in the data and may not generalize well to new data. Random forests help to reduce overfitting by combining the predictions of multiple decision trees.

Robustness to outliers: Decision trees can be sensitive to outliers and missing data, which can lead to a biased model. Random forests are more robust to outliers and missing data, as they consider multiple trees and therefore can better handle noisy data.

Better performance: Random forests can achieve better performance than a single decision tree in terms of accuracy and stability. This is because the randomness introduced in the tree construction process helps to reduce variance and improve the bias-variance tradeoff.

Feature importance: Random forests can provide estimates of the importance of each feature in the data. This can be useful in understanding which features are most predictive of the target variable, and can be used to identify important features for further analysis.

Scalability: Random forests are relatively fast to train and can handle large datasets with many features. This is because the construction of each tree can be done independently and in parallel, making it a good choice for large-scale applications.

11. What is the need of scaling all numerical features in a dataset? Name any two techniques used for scaling.

Scaling is the process of transforming the values of numerical variables to a common scale to ensure that no variable dominates over the others. There are several reasons why scaling is important in data preprocessing:

Improving accuracy of machine learning models: Many machine learning algorithms, such as K-Nearest Neighbors (KNN) and Support Vector

Machines (SVM), are sensitive to the scale of the input features. Scaling the features can improve the accuracy and speed of these models.

Two commonly used techniques for scaling are:

Standardization: In this technique, the values of each feature are transformed to have zero mean and unit variance. This is done by subtracting the mean of the feature from each value and then dividing by the standard deviation. This results in a distribution with a mean of 0 and a standard deviation of 1.

Min-Max Scaling: In this technique, the values of each feature are transformed to a range between 0 and 1. This is done by subtracting the minimum value of the feature from each value and then dividing by the range of the feature (i.e., the difference between the maximum and minimum values). This results in a distribution that ranges from 0 to 1.

12. Write down some advantages which scaling provides in optimization using gradient descent algorithm.

Scaling can provide several advantages when using optimization algorithms, such as Gradient Descent:

Faster convergence: When the input features have significantly different scales, the cost function can have a very elongated shape. This can cause the optimization algorithm to take longer to converge to the minimum. By scaling the features, we can make the cost function more circular, which can help Gradient Descent converge faster.

Avoiding getting stuck in local minima: When the features have different scales, the optimization algorithm can get stuck in a local minimum instead of finding the global minimum. By scaling the features, we can make the optimization algorithm more likely to find the global minimum.

More stable and accurate results: Scaling can help make the optimization process more stable and accurate by reducing the effect of numerical instability. This can help prevent problems such as overflow or underflow, which can occur when working with large or small numbers.

Easier hyperparameter tuning: Scaling can make it easier to tune hyperparameters of the optimization algorithm, such as the learning rate, by making the optimization more stable and predictable.

13. In case of a highly imbalanced dataset for a classification problem, is accuracy a good metric to measure the performance of the model. If not, why?

In the case of a highly imbalanced dataset, accuracy is not a good metric to measure the performance of a model. This is because accuracy is biased towards the majority class, and can be misleading in situations where the classes are imbalanced.

For example, suppose we have a dataset with 95% of the samples belonging to class A and 5% belonging to class B. If we create a model that simply predicts class A for every sample, we would achieve an accuracy of 95%. However, this model is not useful because it does not correctly classify any samples from class B. In this case, accuracy is not a good metric because it does not reflect the performance of the model on the minority class.

14. What is “f-score" metric? Write its mathematical formula.

The F-score, also known as the F1 score, is a measure of a model's accuracy that takes into account both precision and recall. It is a harmonic mean of precision and recall, and provides a single score that represents the balance between the two metrics.

The mathematical formula for the F-score is:

F1 score = 2 * (precision * recall) / (precision + recall)

The F-score ranges from 0 to 1, with a value of 1 indicating perfect precision and recall, and a value of 0 indicating poor performance.

15. What is the difference between fit(), transform() and fit_transform()?

In machine learning, we often need to preprocess the data before feeding it into a model. This can involve tasks such as scaling the data,

encoding categorical variables, or imputing missing values. These preprocessing steps are often implemented using methods from the scikit-learn library, such as fit(), transform(), and fit_transform().

Here's the difference between these three methods:

fit(): The fit() method is used to learn the parameters of the preprocessing step from the training data. For example, if we want to scale the data to have zero mean and unit variance, we would call the fit() method on the training data to learn the mean and standard deviation of each feature.

transform(): The transform() method is used to apply the learned preprocessing step to a new dataset. For example, once we have learned the mean and standard deviation of the features from the training data, we can use the transform() method to apply the same scaling to the test data.

fit_transform(): The fit_transform() method is a shorthand way of calling both the fit() and transform() methods in one step. It learns the parameters of the preprocessing step from the training data and then applies the same preprocessing to the same training data. This method is useful when we have a large dataset and want to apply the same preprocessing to both the training and test data.